

Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Network Security

Lab 04: Firewall Exploration

Supervised by:

Mr. Nasredine LAGRAA

Prepared by:

Smail MERSAD

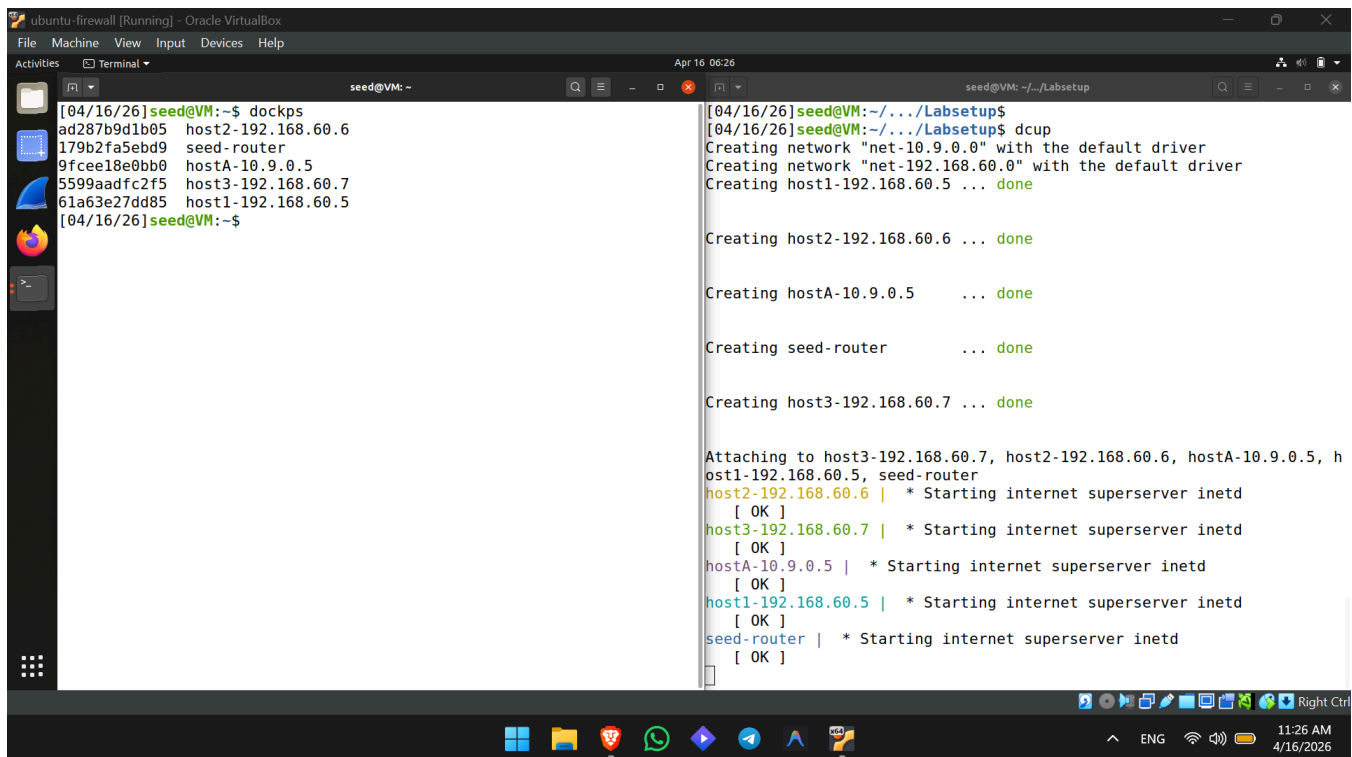
Academic Year: 2025 / 2026

1. Lab Environment Setup

The goal of this lab is to understand how firewalls work by implementing a simple packet-filtering firewall and using iptables . The lab environment consists of an attacker machine, a router, and an internal network with several hosts.

Execution Steps:

1. We navigated to the Labsetup directory on the VM.
2. We built the container images using the provided docker-compose.yml file via the alias \$ dcbuild.
3. We started the containers in the background using \$ dcup.
4. We verified the running containers using \$ dockps.



```
[04/16/26]seed@VM:~$ dockps
ad287b9d1b05   host2-192.168.60.6
179b2fa5ebd9   seed-router
9fcee18e0bb0   hostA-10.9.0.5
5599aadfc2f5   host3-192.168.60.7
61a63e27dd85   host1-192.168.60.5
[04/16/26]seed@VM:~$

[04/16/26]seed@VM:~/../Labsetup$
[04/16/26]seed@VM:~/../Labsetup$ dcbuild
Creating network "net-10.9.0.0" with the default driver
Creating network "net-192.168.60.0" with the default driver
Creating host1-192.168.60.5 ... done

Creating host2-192.168.60.6 ... done

Creating hostA-10.9.0.5 ... done

Creating seed-router ... done

Creating host3-192.168.60.7 ... done

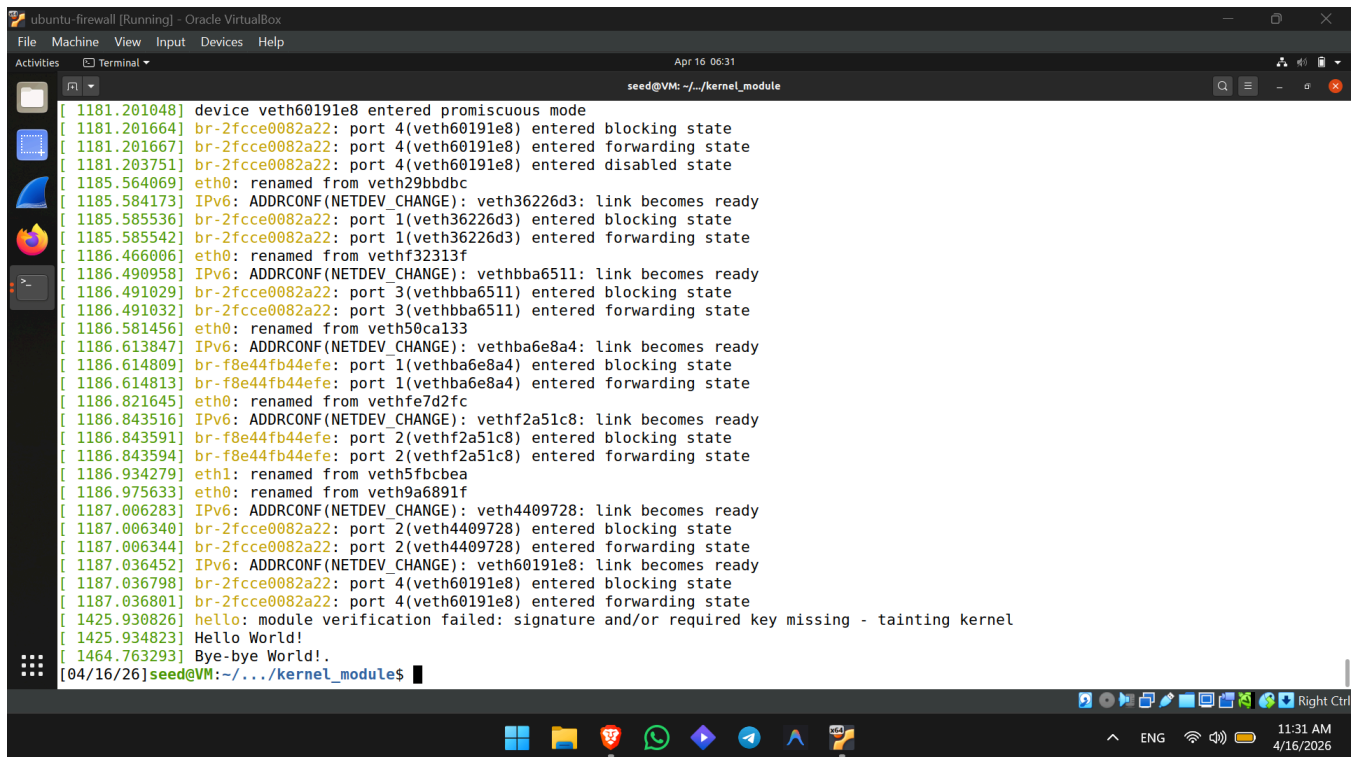
Attaching to host3-192.168.60.7, host2-192.168.60.6, hostA-10.9.0.5, h
ost1-192.168.60.5, seed-router
host2-192.168.60.6 | * Starting internet superserver inetd
[ OK ]
host3-192.168.60.7 | * Starting internet superserver inetd
[ OK ]
hostA-10.9.0.5 | * Starting internet superserver inetd
[ OK ]
host1-192.168.60.5 | * Starting internet superserver inetd
[ OK ]
seed-router | * Starting internet superserver inetd
[ OK ]
```

2. Task 1.A: Implement a Simple Kernel Module

Objective: To become familiar with Loadable Kernel Modules (LKM), which allow us to add new functionalities (like packet filtering) to the Linux kernel at runtime without needing to rebuild the kernel or reboot the computer .

Execution Steps:

1. On the host VM, we located the provided hello.c file and its corresponding Makefile.
2. We compiled the code into a loadable kernel module by running the \$ make command, which generated the hello.ko file.
3. We loaded the module into the kernel using \$ sudo insmod hello.ko.
4. We verified it was loaded by listing the active modules with \$ lsmod | grep hello.
5. We removed the module using \$ sudo rmmod hello.
6. Finally, we checked the kernel logs using \$ dmesg to view the output generated by our module's initialization and cleanup functions .



```
seed@VM: ~/kernel_module
[ 1181.201048] device veth60191e8 entered promiscuous mode
[ 1181.201664] br-2fcce0082a22: port 4(veth60191e8) entered blocking state
[ 1181.201667] br-2fcce0082a22: port 4(veth60191e8) entered forwarding state
[ 1181.203751] br-2fcce0082a22: port 4(veth60191e8) entered disabled state
[ 1185.564069] eth0: renamed from veth29bdbc
[ 1185.584173] IPv6: ADDRCONF(NETDEV_CHANGE): veth36226d3: link becomes ready
[ 1185.585536] br-2fcce0082a22: port 1(veth36226d3) entered blocking state
[ 1185.585542] br-2fcce0082a22: port 1(veth36226d3) entered forwarding state
[ 1186.466006] eth0: renamed from vethf32313f
[ 1186.490958] IPv6: ADDRCONF(NETDEV_CHANGE): vethbba6511: link becomes ready
[ 1186.491029] br-2fcce0082a22: port 3(vethbba6511) entered blocking state
[ 1186.491032] br-2fcce0082a22: port 3(vethbba6511) entered forwarding state
[ 1186.581456] eth0: renamed from veth50ca133
[ 1186.613847] IPv6: ADDRCONF(NETDEV_CHANGE): vethba6e8a4: link becomes ready
[ 1186.614809] br-f8e44fb44efe: port 1(vethba6e8a4) entered blocking state
[ 1186.614813] br-f8e44fb44efe: port 1(vethba6e8a4) entered forwarding state
[ 1186.821645] eth0: renamed from vethfe7d2fc
[ 1186.843516] IPv6: ADDRCONF(NETDEV_CHANGE): vethf2a51c8: link becomes ready
[ 1186.843591] br-f8e44fb44efe: port 2(vethf2a51c8) entered blocking state
[ 1186.843594] br-f8e44fb44efe: port 2(vethf2a51c8) entered forwarding state
[ 1186.934279] eth1: renamed from veth5fbcbea
[ 1186.975633] eth0: renamed from veth9a6891f
[ 1187.006283] IPv6: ADDRCONF(NETDEV_CHANGE): veth4409728: link becomes ready
[ 1187.006340] br-2fcce0082a22: port 2(veth4409728) entered blocking state
[ 1187.006344] br-2fcce0082a22: port 2(veth4409728) entered forwarding state
[ 1187.036452] IPv6: ADDRCONF(NETDEV_CHANGE): veth60191e8: link becomes ready
[ 1187.036798] br-2fcce0082a22: port 4(veth60191e8) entered blocking state
[ 1187.036801] br-2fcce0082a22: port 4(veth60191e8) entered forwarding state
[ 1425.930826] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 1425.934823] Hello World!
[ 1464.763293] Bye-bye World!
[04/16/26]seed@VM:~/kernel_modules$
```

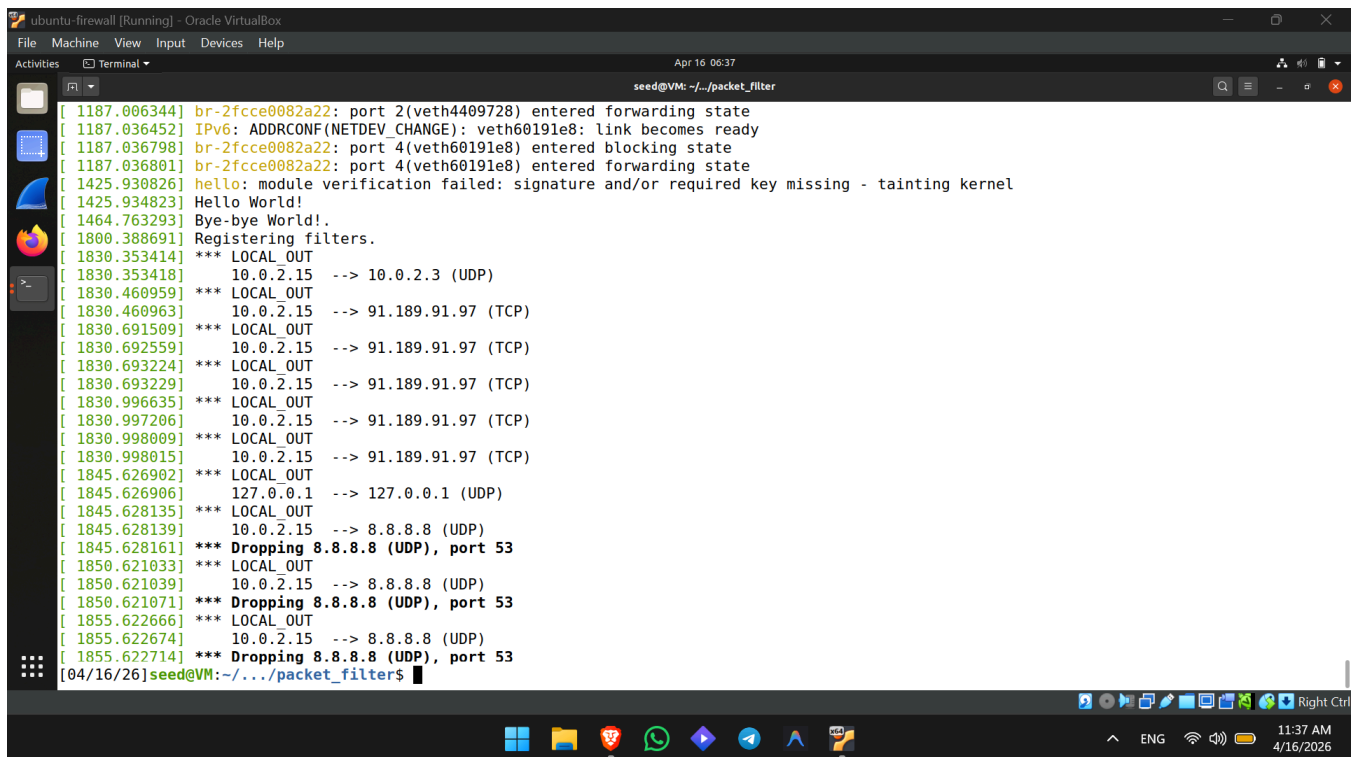
Observation: By observing the kernel logs, we confirmed that our LKM executed successfully. When the module was inserted using insmod, the initialization function was triggered, printing "Hello World!" to the /var/log/syslog file . When the module was removed using rmmod, the cleanup function was triggered, printing "Bye-bye World!". This confirms we can dynamically inject code into the kernel, which is the foundational step for injecting custom firewall rules into the packet processing path.

3. Task 1.B: Implement a Simple Firewall Using Netfilter

Objective: To implement a packet filtering module by hooking functions to Netfilter inside the Linux kernel.

Sub-Task 1: Testing the Sample Code

- **Execution & Observation:** We compiled and loaded the `seedFilter.ko` module. We then executed the command `$ dig @8.8.8.8 www.example.com` to query Google's DNS. The command timed out, and upon checking `dmesg`, we observed the kernel module successfully intercepted and dropped the UDP packet destined for port 53 on 8.8.8.8.

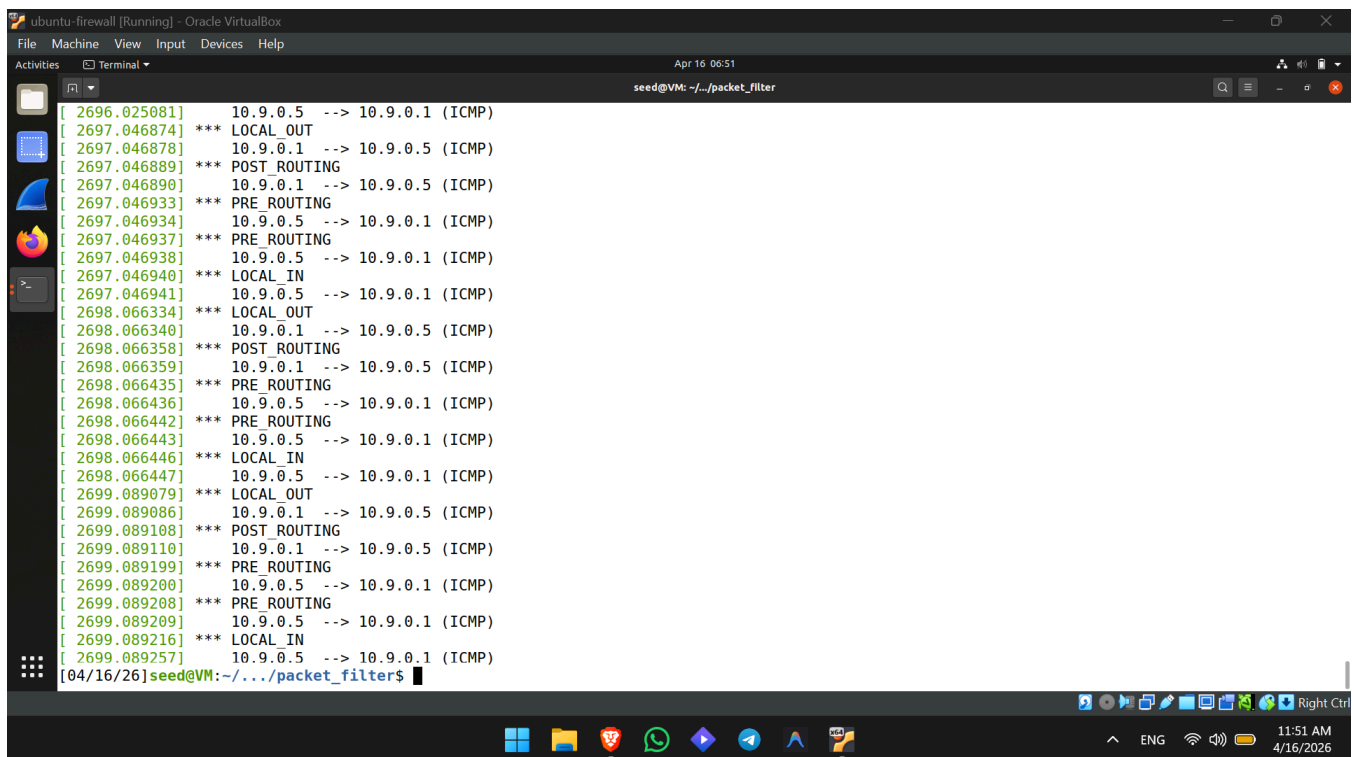


```
ubuntu-firewall [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
seed@VM: ~/../packet_filter

[ 1187.006344] br-2fcce0082a22: port 2(veth4409728) entered forwarding state
[ 1187.036452] IPv6: ADDRCONF(NETDEV_CHANGE): veth60191e8: link becomes ready
[ 1187.036798] br-2fcce0082a22: port 4(veth60191e8) entered blocking state
[ 1187.036801] br-2fcce0082a22: port 4(veth60191e8) entered forwarding state
[ 1425.930826] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 1425.934823] Hello World!
[ 1464.763293] Bye-bye World!.
[ 1800.388691] Registering filters.
[ 1830.353414] *** LOCAL_OUT
[ 1830.353418] 10.0.2.15 --> 10.0.2.3 (UDP)
[ 1830.460959] *** LOCAL_OUT
[ 1830.460963] 10.0.2.15 --> 91.189.91.97 (TCP)
[ 1830.691509] *** LOCAL_OUT
[ 1830.692559] 10.0.2.15 --> 91.189.91.97 (TCP)
[ 1830.693224] *** LOCAL_OUT
[ 1830.693229] 10.0.2.15 --> 91.189.91.97 (TCP)
[ 1830.996635] *** LOCAL_OUT
[ 1830.997206] 10.0.2.15 --> 91.189.91.97 (TCP)
[ 1830.998009] *** LOCAL_OUT
[ 1830.998015] 10.0.2.15 --> 91.189.91.97 (TCP)
[ 1845.626902] *** LOCAL_OUT
[ 1845.626906] 127.0.0.1 --> 127.0.0.1 (UDP)
[ 1845.628135] *** LOCAL_OUT
[ 1845.628139] 10.0.2.15 --> 8.8.8.8 (UDP)
[ 1845.628161] *** Dropping 8.8.8.8 (UDP), port 53
[ 1850.621033] *** LOCAL_OUT
[ 1850.621039] 10.0.2.15 --> 8.8.8.8 (UDP)
[ 1850.621071] *** Dropping 8.8.8.8 (UDP), port 53
[ 1855.622666] *** LOCAL_OUT
[ 1855.622674] 10.0.2.15 --> 8.8.8.8 (UDP)
[ 1855.622714] *** Dropping 8.8.8.8 (UDP), port 53
[04/16/26]seed@VM:~/../packet_filter$
```

Sub-Task 2: Hooking `printInfo` to All Netfilter Hooks

- **Code Modification:** We modified the `seedFilter.c` file to register `printInfo` across all five available Netfilter hooks.
- **Observation & Explanation:** By generating network traffic and analyzing the `dmesg` output, we observed the packet lifecycle through the Netfilter architecture:
 - **NF_INET_PRE_ROUTING:** Invoked for all incoming packets before any routing decisions are made.
 - **NF_INET_LOCAL_IN:** Invoked for incoming packets explicitly destined for the local machine after routing.
 - **NF_INET_FORWARD:** Invoked for incoming packets that are destined for another host (the machine is acting as a router).
 - **NF_INET_LOCAL_OUT:** Invoked for locally generated packets before routing.
 - **NF_INET_POST_ROUTING:** Invoked for all outgoing packets right before they leave the network interface.



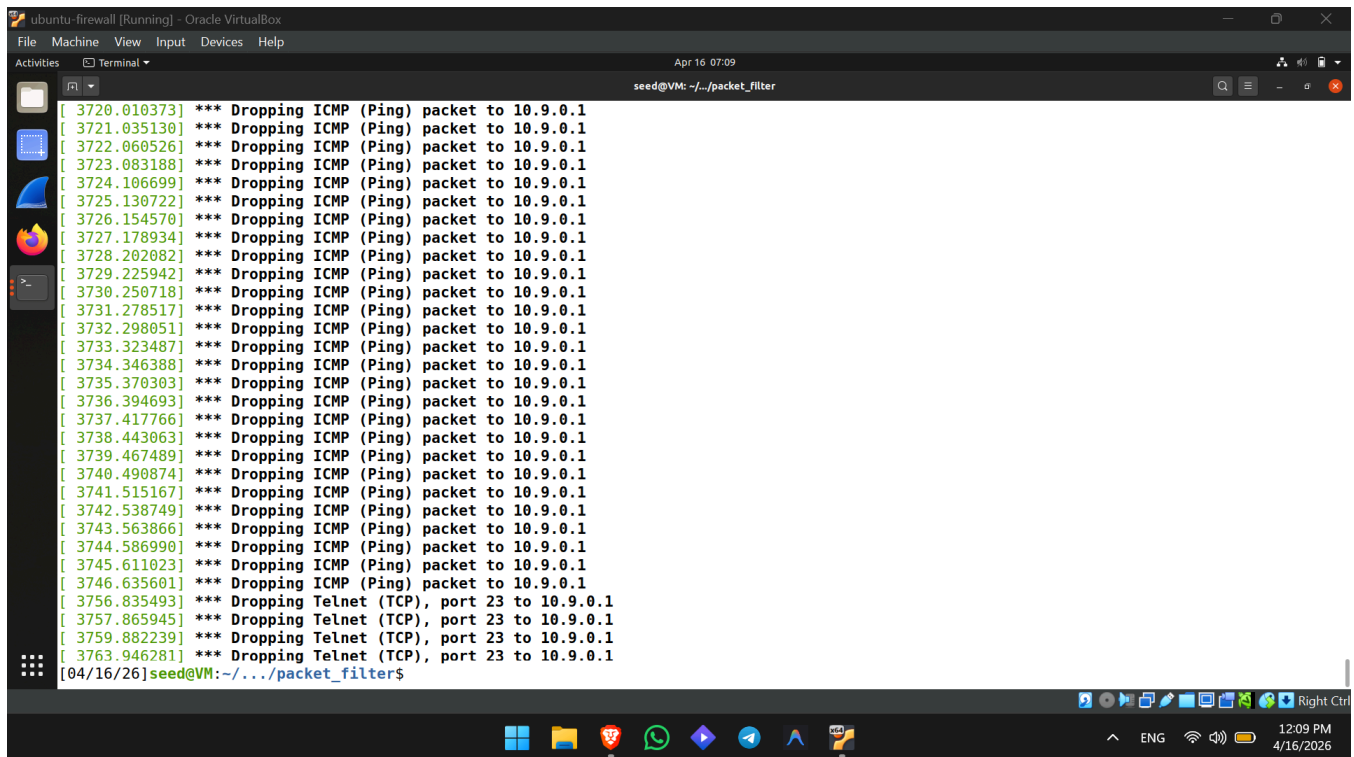
The screenshot shows a terminal window titled "ubuntu: firewall [Running] - Oracle VM". The terminal output displays a series of log messages from the `seedFilter` program, showing the packet lifecycle through Netfilter hooks. The logs are as follows:

```
[ 2696.025081] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2697.046874] *** LOCAL_OUT
[ 2697.046878] 10.9.0.1 --> 10.9.0.5 (ICMP)
[ 2697.046889] *** POST_ROUTING
[ 2697.046890] 10.9.0.1 --> 10.9.0.5 (ICMP)
[ 2697.046933] *** PRE_ROUTING
[ 2697.046934] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2697.046937] *** PRE_ROUTING
[ 2697.046938] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2697.046940] *** LOCAL_IN
[ 2697.046941] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2698.066334] *** LOCAL_OUT
[ 2698.066340] 10.9.0.1 --> 10.9.0.5 (ICMP)
[ 2698.066358] *** POST_ROUTING
[ 2698.066359] 10.9.0.1 --> 10.9.0.5 (ICMP)
[ 2698.066435] *** PRE_ROUTING
[ 2698.066436] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2698.066442] *** PRE_ROUTING
[ 2698.066443] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2698.066446] *** LOCAL_IN
[ 2698.066447] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2699.089079] *** LOCAL_OUT
[ 2699.089086] 10.9.0.1 --> 10.9.0.5 (ICMP)
[ 2699.089108] *** POST_ROUTING
[ 2699.089110] 10.9.0.1 --> 10.9.0.5 (ICMP)
[ 2699.089199] *** PRE_ROUTING
[ 2699.089200] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2699.089208] *** PRE_ROUTING
[ 2699.089209] 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 2699.089216] *** LOCAL_IN
[ 2699.089257] 10.9.0.5 --> 10.9.0.1 (ICMP)
[04/16/26] seed@VM: ~/.../packet_filter$
```

The terminal window also shows the system clock as "Apr 16 06:51" and the user prompt "seed@VM: ~/.../packet_filter\$". The bottom of the window shows the Ubuntu desktop environment with various application icons and a system tray.

Sub-Task 3: Blocking Ping and Telnet

- **Code:** *[Insert the C code snippets for your ICMP blocking function and TCP blocking function here]*
- **Execution & Observation:** We implemented two custom hook functions to block ICMP echo requests and TCP connections targeting port 23. We registered both to the `NF_INET_LOCAL_IN` hook because these packets are destined for the host machine itself (10.9.0.1). To test, we accessed the `10.9.0.5` container and executed `$ ping 10.9.0.1` and `$ telnet 10.9.0.1`. Both requests hung indefinitely and failed. Checking the host's `dmesg` confirmed our custom Netfilter hooks actively intercepted and dropped the respective packets, demonstrating a successfully implemented stateless firewall within the kernel.



```
ubuntu-firewall [Running] - Oracle VMVirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 16 07:09
seed@VM: ~/.../packet_filter

[ 3720.010373] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3721.035130] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3722.060526] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3723.083188] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3724.106699] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3725.130722] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3726.154570] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3727.178934] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3728.202082] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3729.225942] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3730.250718] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3731.278517] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3732.298051] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3733.323487] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3734.346388] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3735.370303] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3736.394693] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3737.417766] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3738.443063] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3739.467489] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3740.490874] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3741.515167] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3742.538749] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3743.563866] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3744.586990] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3745.611023] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3746.635601] *** Dropping ICMP (Ping) packet to 10.9.0.1
[ 3756.835493] *** Dropping Telnet (TCP), port 23 to 10.9.0.1
[ 3757.865945] *** Dropping Telnet (TCP), port 23 to 10.9.0.1
[ 3759.882239] *** Dropping Telnet (TCP), port 23 to 10.9.0.1
[ 3763.946281] *** Dropping Telnet (TCP), port 23 to 10.9.0.1
[04/16/26]seed@VM:~/.../packet_filter$
```

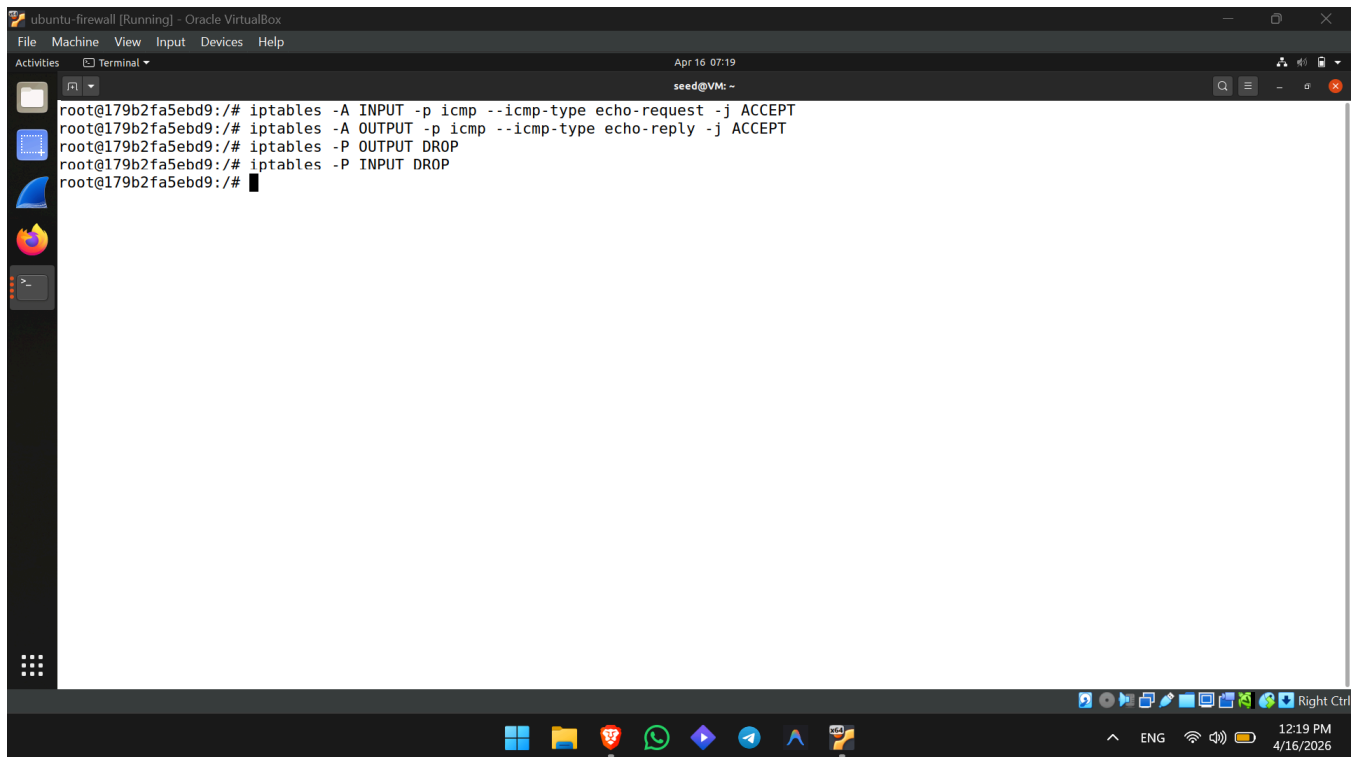
4. Task 2.A: Protecting the Router

Objective: To configure iptables on the Router machine to block all incoming connections except for ICMP echo requests (ping) .

Commands and Explanations:

We executed the following rules on the Router container:

- iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT: This appends (-A) a rule to the INPUT chain. It targets the ICMP protocol (-p icmp) specifically looking for ping requests (--icmp-type echo-request) and tells the firewall to accept them (-j ACCEPT).
- iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT: This appends a rule to the OUTPUT chain to allow the router to send ICMP ping replies back to the requester.
- iptables -P OUTPUT DROP: This changes the default policy (-P) of the OUTPUT chain to DROP. If an outgoing packet doesn't match any specific rule above, it will be dropped .
- iptables -P INPUT DROP: This changes the default policy of the INPUT chain to DROP. Any incoming packet that isn't explicitly accepted by a rule is dropped.



The screenshot shows a terminal window titled 'ubuntu: firewall [Running] - Oracle VirtualBox'. The terminal output displays the following commands and their execution:

```
root@179b2fa5ebd9:/# iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
root@179b2fa5ebd9:/# iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT
root@179b2fa5ebd9:/# iptables -P OUTPUT DROP
root@179b2fa5ebd9:/# iptables -P INPUT DROP
root@179b2fa5ebd9:/#
```

The terminal window is part of a desktop environment with a taskbar at the bottom showing various application icons and system status indicators like time (12:19 PM) and date (4/16/2026).

```
ubuntu: firewall [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
seed@VM: ~
[04/16/26]seed@VM:~$ dockps
ad287b9d1b05 host2-192.168.60.6
179b2fa5ebd9 seed-router
9fcee18e0bb0 hostA-10.9.0.5
5599aadfc2f5 host3-192.168.60.7
61a63e27dd85 host1-192.168.60.5
[04/16/26]seed@VM:~$ docksh 9fcee18e0bb0
root@9fcee18e0bb0:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.475 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.167 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.171 ms
^C
--- 10.9.0.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2050ms
rtt min/avg/max/mdev = 0.167/0.271/0.475/0.144 ms
root@9fcee18e0bb0:/# telnet 10.9.0.11
Trying 10.9.0.11...
^C
root@9fcee18e0bb0:/#
```

Observations: We tested the firewall from the 10.9.0.5 host.

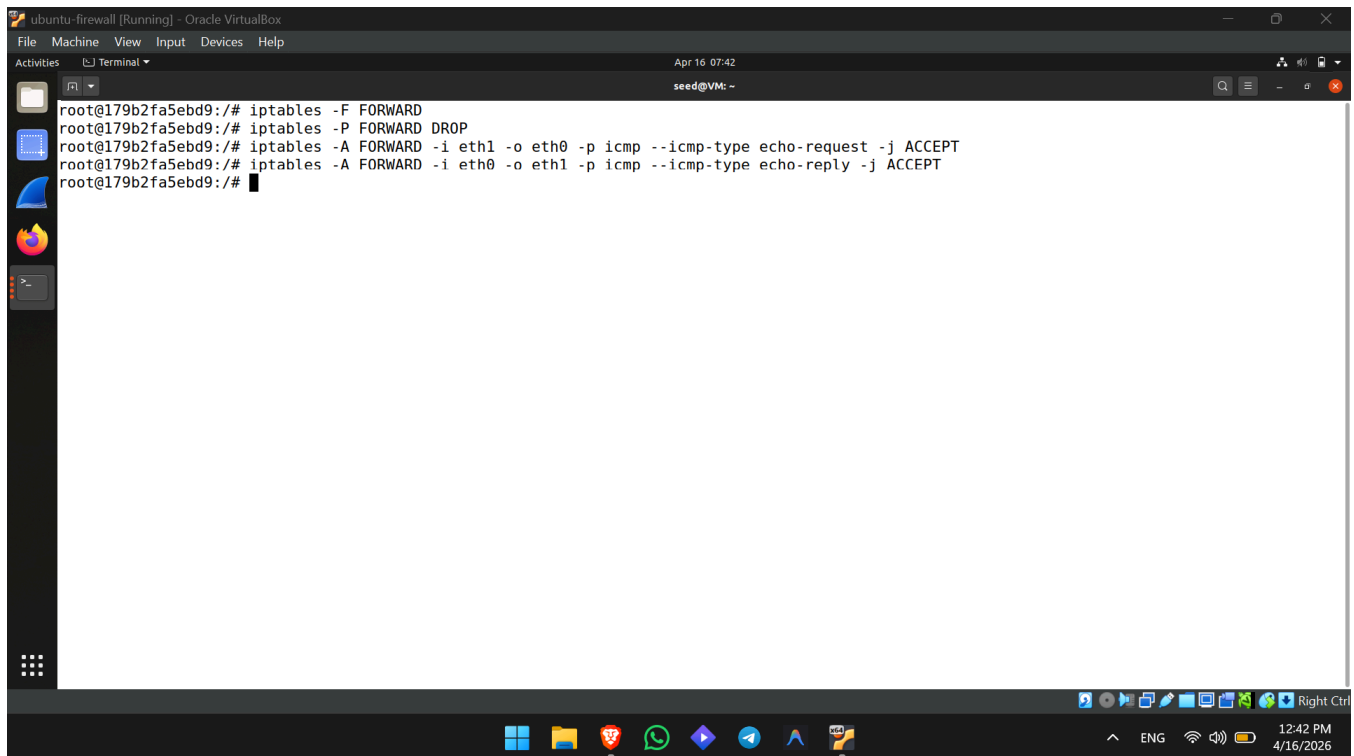
1. **Can you ping the router?** Yes. The ping succeeded because our first two explicit rules caught the ICMP echo requests and replies, allowing them through before the default DROP policy could act on them .
2. **Can you telnet into the router?** No. The telnet connection timed out. Because we did not add a specific rule to accept TCP packets on port 23, the telnet packets fell through to the default INPUT DROP policy and were discarded by the firewall .

5. Task 2.B: Protecting the Internal Network

Objective: To implement firewall rules on the Router using the FORWARD chain to protect the 192.168.60.0/24 internal network while allowing specific outgoing ICMP traffic .

Commands and Explanations: First, we identified our network interfaces using ip addr: eth0 for the external network and eth1 for the internal network . We then applied the following rules on the Router:

1. iptables -P FORWARD DROP: This sets the default policy for the FORWARD chain to drop all packets passing through the router. This explicitly satisfies Objective 4 (blocking all other packets between networks). By defaulting to DROP, this also satisfies Objective 1, as external pings to internal hosts will be blocked without an explicit allow rule.
2. iptables -A FORWARD -i eth1 -o eth0 -p icmp --icmp-type echo-request -j ACCEPT: This rule permits ICMP echo requests originating from the internal interface (eth1) and heading to the external interface (eth0) .
3. iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-reply -j ACCEPT: This rule allows the corresponding ICMP echo replies to return from the external network back into the internal network. Together with the previous rule, this satisfies Objective 3.



```
ubuntu-firewall [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 16 07:42
seed@VM: ~
root@179b2fa5ebd9:/# iptables -F FORWARD
root@179b2fa5ebd9:/# iptables -P FORWARD DROP
root@179b2fa5ebd9:/# iptables -A FORWARD -i eth1 -o eth0 -p icmp --icmp-type echo-request -j ACCEPT
root@179b2fa5ebd9:/# iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-reply -j ACCEPT
root@179b2fa5ebd9:/#
```

```
ubuntu-firewall [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 16 07:41
seed@VM: ~

root@9fcee18e0bb0:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
21 packets transmitted, 0 received, 100% packet loss, time 20472ms

root@9fcee18e0bb0:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.164 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.151 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.992 ms
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.163 ms
^C
--- 10.9.0.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3060ms
rtt min/avg/max/mdev = 0.151/0.367/0.992/0.360 ms
root@9fcee18e0bb0:/#
```

```
ubuntu-firewall [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal
Apr 16 07:41
seed@VM: ~

root@61a63e27dd85:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=63 time=0.647 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=63 time=0.170 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=63 time=0.248 ms
64 bytes from 10.9.0.5: icmp_seq=4 ttl=63 time=0.182 ms
^C
--- 10.9.0.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.170/0.311/0.647/0.195 ms
root@61a63e27dd85:/#
```

Observations: We validated our firewall using ping tests from both sides of the network:

- **Outside hosts cannot ping internal hosts:** A ping from 10.9.0.5 to 192.168.60.5 failed because the FORWARD default policy dropped the echo-request packets.
- **Outside hosts can ping the router:** A ping from 10.9.0.5 to 10.9.0.11 succeeded. This is because packets destined *for* the router traverse the INPUT chain, not the FORWARD chain, and our INPUT policy was reset to ACCEPT.
- **Internal hosts can ping outside hosts:** A ping from 192.168.60.5 to 10.9.0.5 succeeded because our explicit rules allowed the echo-request out and the echo-reply back in.